# Session 02:

## Java 8 (Phần 1)

RIKKEI Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# NỘI DUNG

1. Giới thiệu chung về Java 8
2. Tính năng mới của Interface từ Java 8
3. Functional Interface và biểu thức Lambda
4. Tham chiếu phương thức (Method Reference)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 1. Giới thiệu chung về Java 8

- **Cột mốc quan trọng:** 2014 đánh dấu cột mốc quan trọng đối với Java khi Java 8 ra đời
- **Thay đổi tư duy:** Chuyển từ lập trình hướng đối tượng (OOP) thuần túy sang kết hợp **Lập trình hướng hàm (Functional Programming)**
- **Long-Term Support:** Java 8 là phiên bản hỗ trợ dài hạn nhất, dự kiến đến năm 2030

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 1. Giới thiệu chung về Java 8

## Sự bắt kịp và tiếp thu

- **Bối cảnh:**
  - Sự xuất hiện của ngôn ngữ lập trình hướng hàm (**Functional Programming**) với cú pháp ngắn gọn: **"Write less, do more"**
- **Hành động:**
  - Java 8 tiếp thu và hỗ trợ phong cách viết giống lập trình hướng hàm, ngắn gọn hơn, dễ hiểu hơn

***"Write less, do more"***

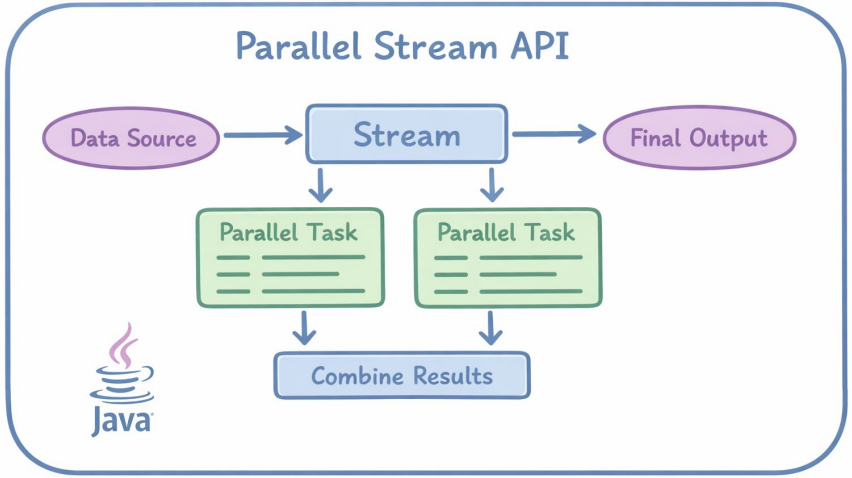
```java
List<String> list = new ArrayList<String>(List.of("Son", "Hung", "Mai"));
String surName = " Do";
// Trước Java 8
for (String item : list) {
    if (item.equals("Hung"))
        continue;
    item += surName;
    System.out.println(item);
}
// Java 8
list.stream()
    .filter(i -> !i.equals("Hung"))
    .map(i -> i + surName)
    .forEach(System.out::println);
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 1. Giới thiệu chung về Java 8

## Sự bắt kịp và tiếp thu

- **Bối cảnh:**
  - CPU của máy tính đang chuyển dần sang tăng số lượng nhân (multi-core) thay vì tăng tốc độ đơn nhân
- **Hành động:**
  - Java 8 cung cấp API xử lý đa luồng hiệu quả với tính năng Stream API:
    - parallelStream()




© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 1. Giới thiệu chung về Java 8

## Các tính năng cốt lõi

- **Lambda Expressions:** Biểu diễn phương thức của Interface dưới dạng biểu thức ngắn gọn
- **Stream API:** Xử lý tập hợp dữ liệu theo theo kiểu lập trình hướng hàm trên các luồng phần tử (**filter, map, reduce**)
- **Optional:** Giải pháp triệt tiêu lỗi **NullPointerException**
- **Date/Time API:** Giới thiệu các lớp cốt lõi trong gói **java.time** sử dụng **hệ thống lịch định nghĩa trong ISO-8601** (lịch **Gregorian**), các **hệ thống lịch không thuộc ISO** biểu diễn bằng **java.time.chrono** (lịch **Hijrah**, **Phật giáo**)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 2. Tính năng mới của Interface từ Java 8

## Phương thức mặc định (Default method)

- **Vấn đề:** Trước Java 8, thêm phương thức vào Interface sẽ làm hỏng các lớp đang thực thi nó
- **Giải pháp:** Sử dụng từ khóa **default** để định nghĩa phương thức có thân hàm ngay trong **Interface**

```java
interface User {
    void login();
    default boolean isAdmin() { return false; }
    default void authenticate() {
        System.out.println("User authenticated");
    }
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 2. Tính năng mới của Interface từ Java 8

## Phương thức mặc định (Default method)

- **Đặc điểm:** Được kế thừa như method của class, các lớp thực thi không bắt buộc phải ghi đè (override) nhưng có thể ghi đè nếu cần.

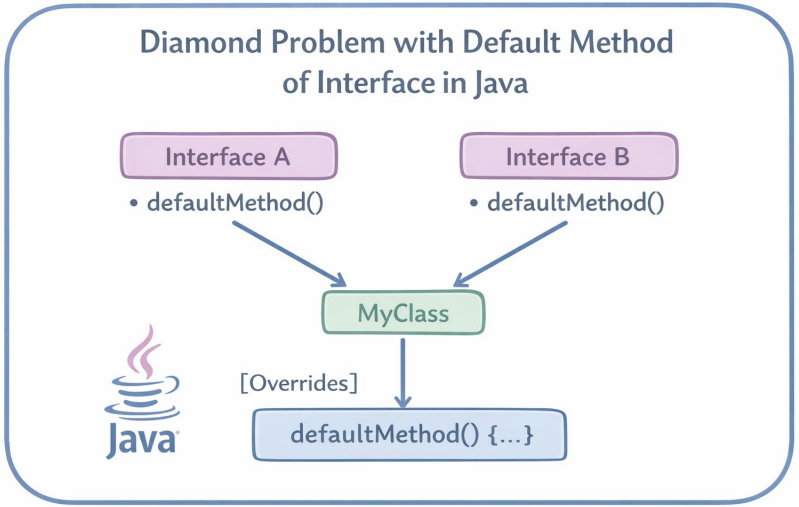
```java
public class Admin implements User {
    public void login() {
        System.out.println("Admin logged in");
    }
    @Override
    public boolean isAdmin() {
        return true;
    }
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 2. Tính năng mới của Interface từ Java 8

## Phương thức mặc định (Default method)

- **Lưu ý:** Vấn đề **xung đột phương thức mặc định**: **Diamond Problem**




```java
public class Admin implements User, AdminAuth {
    public void login() {
        System.out.println("Admin logged in");
    }
    @Override
    public boolean isAdmin() {
        return true;
    }
    @Override
    public void authenticate() {
        AdminAuth.super.authenticate();
    }
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 2. Tính năng mới của Interface từ Java 8

## Phương thức tĩnh (Static method)

- **Đặc điểm:**
  - Không thể bị ghi đè (non-overridable)
  - Chỉ có thể gọi thông qua tên của Interface
- **Mục đích:** Cung cấp các phương thức tiện ích (helper methods) liên quan chặt chẽ đến Interface đó.

```java
interface MathUtil {
    static int square(int x) {
        return x * x;
    }
}
int result = MathUtil.square(5);
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 2. Tính năng mới của Interface từ Java 8

## So sánh phương thức mặc định và phương thức tĩnh

| Tiêu chí | Phương thức mặc định | Phương thức tĩnh |
|---|---|---|
| Có thể được kế thừa | Có, tương tự như method của class. | Không. |
| Có thể được override | Có, và logic của lớp triển khai được ưu tiên. | Không. |
| Gọi qua object | Có. | Không. |

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Functional Interface

- **Khái niệm:**
  - Là **Interface chỉ có duy nhất một phương thức trừu tượng**.
  - Bất kì Interface nào tuân thủ **SAM** - **Single Abstract Method**, đều được coi là **Functional Interface**.
  - Có thể chứa nhiều **phương thức default** hoặc **static**.
  - Annotation **@FunctionalInterface** là không bắt buộc, nó chỉ dùng để hỗ trợ compile-time.

```java
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
    static int square(int a) {
        return a * a;
    }
    default double pi() {
        return Math.PI;
    }
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Functional Interface

- Sự ra đời của **Functional Interface** là vô cùng quan trọng, bởi vì:
  - Java 8 bổ sung biểu thức Lambda, mà Lambda cần một "đích" (target type).
  - Lambda cần biết nó đang triển khai (implement) cho phương thức trừu tượng nào, thế nên **Functional Interface** chỉ được phép có một phương thức trừu tượng.

```
Calculator add = (a, b) -> a + b;
add.calculate(10, 5); // 15
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Biểu thức Lambda (Lambda Expression)

- **Lambda** cho phép viết mã một cách ngắn gọn hơn bằng cách biểu diễn các **hàm ẩn danh (anonymous function)**.
- Nhờ Lambda, ta có thể:
  - Truyền một đoạn mã như một tham số cho phương thức.
  - Hoặc gán nó vào biến.

```
(int x, int y) -> { return x + y; }
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Biểu thức Lambda (Lambda Expression)

- **Cú pháp Lambda:**

```
(int x, int y) -> { return x + y; }
```

- **Danh sách tham số ( ):**
  - Nếu chỉ 1 tham số, có thể bỏ ().
  - Nếu khai báo kiểu dữ liệu, bắt buộc dùng ().
- **Toán tử Lambda ->:** Phân cách tham số và thân hàm.
- **Thân Lambda { }:**
  - Có thể là 1 hoặc nhiều câu lệnh.
  - Nếu nhiều câu lệnh, bắt buộc dùng {}.
  - Nếu chỉ trả về giá trị ngay, có thể bỏ {} và return.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Biểu thức Lambda (Lambda Expression)

- **Các cú pháp phổ biến:**

Không tham số:

```
() → System.out.println("Hello World");
```

Một tham số:

```
x → x * x
```

Hai tham số:

```
(a, b) → a + b
```

Có thân hàm:

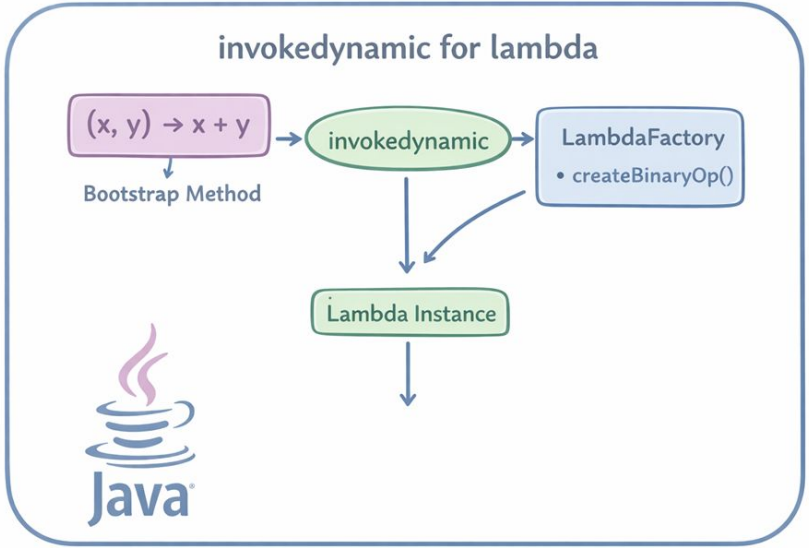
```
(a, b) → {
    int sum = a + b;
    return sum;
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Biểu thức Lambda (Lambda Expression)

- **Cơ chế của Lambda trong JVM:**
  - **Bản chất:** Lambda không tạo ra lớp ẩn danh (**.class**) theo cách truyền thống
  - **Cơ chế:** JVM sử dụng lệnh **invokedynamic** để tạo implementation động tại thời điểm thực thi
  - **Ý nghĩa:** Tối ưu hóa hiệu năng và giảm bớt gánh nặng cho bộ nhớ so với cách dùng anonymous class cũ




© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Các Functional Interface phổ biến

| Tên Interface | Tham số nhận vào | Kết quả trả về | Mục đích sử dụng |
|---|---|---|---|
| **Predicate** | 1 giá trị | boolean | Kiểm tra điều kiện (filter) |
| **Function<T, R>** | 1 giá trị | 1 giá trị khác | Biến đổi dữ liệu (map) |
| **Consumer** | 1 giá trị | void | Xử lý dữ liệu (in ấn, lưu trữ) |
| **Supplier** | Không có | 1 giá trị | Dùng khi cần tạo dữ liệu |

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 3. Functional Interface và biểu thức Lambda

## Ví dụ với Functional Interface Comparator

- **Bối cảnh:** Comparator đã có từ lâu, nhưng từ Java 8, được cải tiến thành **Functional Interface**
- **Ứng dụng:** Phương thức sort() nhận một Lambda làm "công thức" so sánh thay vì một đối tượng cồng kềnh
- **Lợi ích:** Tách biệt rõ ràng giữa dữ liệu và cách xử lý dữ liệu

```java
// Trước Java 8
names.sort(new Comparator<String>() {
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});
// Java 8
names.sort((a, b) -> a.compareTo(b));
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 4. Tham chiếu phương thức (Method References)

- **Khái niệm:** Là cú pháp rút gọn tối đa của Lambda khi Lambda đó chỉ gọi một phương thức đã tồn tại
- **Toán tử đặc trưng: ::**
- **Ví dụ:**

```
list.forEach((item) -> System.out.println(item));
// Method reference
list.forEach(System.out::println);
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 4. Tham chiếu phương thức (Method References)

- **Phân loại 4 kiểu Tham chiếu phương thức:**
- **Static Method:** Class::staticMethod (Ví dụ: Integer::parseInt)
- **Instance Method (Object):** obj::instanceMethod (Ví dụ: System.out::println)
- **Instance Method (Class):** Class::instanceMethod (Ví dụ: String::equals)
- **Constructor:** Class::new (Ví dụ: ArrayList::new)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# 4. Tham chiếu phương thức (Method References)

- **Phân loại 4 kiểu Tham chiếu phương thức:**

| Loại | Cú pháp | Lambda tương đương |
|---|---|---|
| **Static Method** | Class::staticMethod | (args) -> Class.staticMethod(args) |
| **Instance Method (Object)** | obj::instanceMethod | (args) -> object.method(args) |
| **Instance Method (Class)** | Class::instanceMethod | (obj,args) -> obj.method(args) |

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

# TỔNG KẾT

- Ý nghĩa cốt lõi: Java 8 là nền tảng cho lập trình hiện đại, kết hợp hài hòa OOP và Functional Programming
- Tầm quan trọng: Là điều kiện tiên quyết để học các API nâng cao như Stream API và xử lý song song
- Thông điệp: "Viết ít hơn, làm được nhiều hơn" (Write less, do more)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

# KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN